

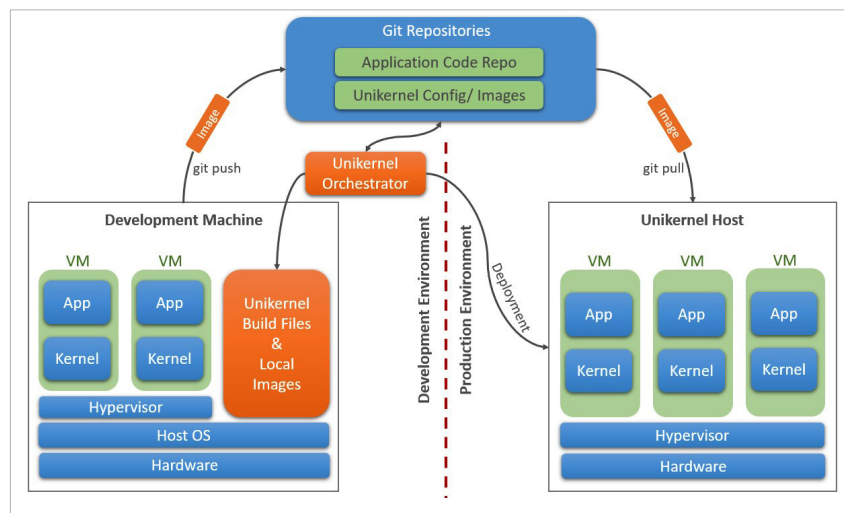


Figure 2: Unikernel ecosystem

Hypervisors	Public Cloud Providers
KVM	Google Cloud
Xen	Amazon Web Services
OpenStack	Microsoft Azure
ProxMox	Oracle Cloud
ESX	Digital Ocean
FireCracker	Vultr
VirtualBox	UpCloud
Hyper-V	

Figure 3: Supported platforms

to build standalone environments like QA or staging, and run tests against them before deploying the application to production. This approach will not require us to manage or configure the environment beforehand.

Where to use unikernels

While unikernels can be primarily used in cloud computing, they can also be used in Internet of Things (IoT) and networking devices. These platforms will benefit from unikernels' flexible, lightweight and scalable architecture. Faster boot times allow faster recovery of failed services or network nodes, and on the flip side, spin up VMs faster in an autoscaled environment thus increasing availability. IoT devices with hardware limitations can benefit from unikernels as they have a smaller footprint and are

highly optimised. Most of these devices do not require a mutable system.

Unikernels are ideal for the deployment of microservices and other transient services. Microservices are now quite popular -- they break the system into several individual and fully decoupled or disjointed services, rather than packing all the features of an application into a large monolithic program. Unikernels, being single-threaded by default, are well suited for this kind of architecture. A key benefit of unikernels concerning microservices is that each VM is completely self-contained and isolated. This provides total protection from system-wide breaches — if a failure occurs in any part of the system it can recover fast without affecting other services. This increases the overall resiliency and security of the system.

Imagine an application with 15 different microservices running from 15 small unikernel instances with the potential to launch more instances if one fails or if autoscaling happens. These instances are all standalone VMs, which do not require any OS administration, vulnerability remediation, and no additional overhead of a hypervisor or host operating system that our application is not even using. There are unikernel projects which can run on Kubernetes, so scheduling and clustering are possible.

Unikernels can also be used for hosting applications that perform a specific task in response to an event and are terminated after that. Think about cloud functions like AWS Lambda or Azure Functions. There can be concerns with data storage as unikernels are read-only, remember? However, this limitation can be easily overcome as most unikernels support mounting volumes, as in Docker.

There are a lot of other use cases where unikernels will fit in, if not today, maybe in the near future. As Ronald J. Garan Jr, a NASA astronaut, once said, "We are limited only by our imagination and our will to act."

Presenting Nanos

Nanos is a legacy unikernel, which means it can run applications developed in a broad range of programming languages. Nanos is free open source software; the source code is freely available on GitHub, which can be compiled or customised as per our requirement. Currently, Nanos supports X86-64 and ARM64 but has limited RISC-V support. It can be deployed as a guest VM directly on top of the hypervisors or virtualisation platforms and public cloud providers listed in Figure 3.

Nanos unikernels also run on Kubernetes, thus providing support for clustering and orchestration of unikernels on multiple hosts. To know more about Nanos from a technical standpoint, you can read the book available at <https://Nanos.org/thebook>.

From an application perspective, Nanos has all the advantages and features of unikernels. Additionally, it has lots of advanced security implementations which other unikernels may not have, along with its ease of setup with Ops. Ops is a tool, also known as an orchestrator, used to package, create, deploy and run applications as a Nanos unikernel instance. That being said, let's now deploy our demo app with Nanos using Ops.